تقرير إعادة هيكلة نظام إدارة المكتبة وفق مبادئ SOLID

المقدمة

يهدف هذا التقرير إلى توثيق عملية إعادة هيكلة (Refactoring) نظام إدارة المكتبة الذي تم تطويره في الأسبوع الثالث، وذلك لتطبيق مبادئ SOLID الخمسة في البرمجة كائنية التوجه (Object-Oriented Programming). تهدف هذه المبادئ إلى تصميم أنظمة برمجية مرنة، قابلة للصيانة، وقابلة للتوسع.

1. مبدأ المسؤولية الواحدة (Single Responsibility Principle - SRP)

الشرح

ينص مبدأ المسؤولية الواحدة على أن **"يجب أن يكون للكلاس سبب واحد فقط للتغيير"**. بمعنى آخر، يجب أن يكون الكلاس مسؤولاً عن وظيفة واحدة محددة جيداً. إذا كان الكلاس يقوم بأكثر من وظيفة، فإنه ينتهك هذا المبدأ ويصبح أكثر صعوبة في الصيانة والتعديل.

الكود الأصلي (قبل التعديل)

في نظام الأسبوع الثالث، كان الكلاس الرئيسي للمكتبة (أو حتى `Program.cs` الذي يحتوي على منطق المكتبة) مسؤولاً عن عدة مهام مثل:

- إدارة عناصر المكتبة (إضافة، حذف، بحث).
- إدارة الأعضاء.
- معالجة عمليات الاستعارة والإرجاع.
- عرض المخرجات للمستخدم.
- قراءة المدخلات من المستخدم.

هذا يؤدي إلى كلاس كبير ومعقد يصعب فهمه وتعديله.

C#

```
// مثال مبسط على الكود الأصلي الذي ينتهك SRP
// (الكود الفعلي كان موزعا بشكل مختلف لكن بنفس المخالفة)
public class Library
{
    private List<LibraryItem> _items = new List<LibraryItem>();
    private List<Member> _members = new List<Member>();

    public void AddItem(LibraryItem item) { /* ... */ }
```

```
    public void RemoveItem(string isbn) { /* ... */ }
    public LibraryItem FindItem(string isbn) { /* ... */ }

    public void AddMember(Member member) { /* ... */ }
    public Member FindMember(int memberId) { /* ... */ }

    public void BorrowItem(string isbn, int memberId) { /* ... */ }
    public void ReturnItem(string isbn, int memberId) { /* ... */ }

    public void DisplayMenu() { /* ... */ }
    public string GetUserInput() { /* ... */ }
}
```

المشكلة/المخالفة

الكلاس `Library` (أو `Program.cs` في حالتنا الأصلية) كان مسؤولاً عن كل شيء تقريباً. أي تغيير في طريقة إدارة العناصر، أو الأعضاء، أو عمليات الاستعارة، أو حتى طريقة عرض البيانات، كان سيتطلب تعديل نفس الكلاس. هذا يزيد من احتمالية حدوث الأخطاء ويجعل الكود صعب الصيانة.

الكود بعد التعديل (تطبيق SRP)

تم تقسيم الكلاس الأصلي إلى عدة كلاسات متخصصة، كل منها مسؤول عن وظيفة واحدة:

- **`LibraryCatalog`** : مسؤول عن إدارة عناصر المكتبة (إضافة، حذف، بحث، استرجاع).
- **`MemberService`** : مسؤول عن إدارة أعضاء المكتبة (إضافة، بحث، استرجاع).
- **`BorrowingService`** : مسؤول عن منطق استعارة وإرجاع العناصر.
- **`DisplayService`** : مسؤول عن عرض جميع المخرجات على الكونسول.
- **`InputHelper`** : مسؤول عن قراءة جميع المدخلات من المستخدم.

C#

```
// LibraryCatalog.cs
namespace LibraryManagementApp.Services
{
    public class LibraryCatalog
    {
        private List<LibraryItem> _items;
        public void AddItem(LibraryItem item) { /* ... */ }
        public List<LibraryItem> SearchItems(string query) { /* ... */ }
        public List<LibraryItem> GetAllItems() { /* ... */ }
    }
}

// MemberService.cs
namespace LibraryManagementApp.Services
```

```
{
    public class MemberService
    {
        private List<Member> _members;
        public void AddMember(Member member) { /* ... */ }
        public Member FindMemberById(int memberId) { /* ... */ }
    }
}

// BorrowingService.cs
namespace LibraryManagementApp.Services
{
    public class BorrowingService
    {
        private LibraryCatalog _catalog;
        private MemberService _memberService;
        // ... (يعتمد على خدمات أخرى)
        public void BorrowItem(string isbn, int memberId) { /* ... */ }
        public void ReturnItem(string isbn, int memberId) { /* ... */ }
    }
}

// DisplayService.cs
namespace LibraryManagementApp.Services
{
    public class DisplayService
    {
        public void DisplayItemDetails(LibraryItem item) { /* ... */ }
        public void DisplayMessage(string message) { /* ... */ }
    }
}

// InputHelper.cs
namespace LibraryManagementApp.Services
{
    public class InputHelper
    {
        public string GetStringInput(string prompt) { /* ... */ }
        public int GetIntInput(string prompt) { /* ... */ }
    }
}
```

التحسينات

- **سهولة الصيانة:** أصبح كل كلاس أصغر وأسهل في الفهم والتعديل. أي تغيير في منطق البحث مثلاً، سيؤثر فقط على `LibraryCatalog` .

- **قابلية الاختبار:** يمكن اختبار كل كلاس بشكل مستقل، مما يسهل عملية اكتشاف الأخطاء.
- **الوضوح:** أصبح الكود أكثر وضوحاً، حيث يعبر اسم كل كلاس عن مسؤوليته بوضوح.

2. مبدأ الفتح/الغلق (Open/Closed Principle - OCP)

الشرح

ينص مبدأ الفتح/الغلق على أن **"يجب أن تكون الكيانات البرمجية (الكلاسات، الوحدات، الدوال) مفتوحة للتوسع، ولكن مغلقة للتعديل"**. هذا يعني أنه يجب أن نتمكن من إضافة وظائف جديدة للنظام دون الحاجة لتعديل الكود الموجود الذي يعمل بالفعل.

الكود الأصلي (قبل التعديل)

في نظام الأسبوع الثالث، إذا أردنا إضافة نوع جديد من عناصر المكتبة (مثل `Magazine`)، كان من الممكن أن نضطر لتعديل الكلاسات التي تتعامل مع `LibraryItem` بشكل مباشر، أو إضافة شروط `if-else` أو `switch` في أماكن متعددة للتعامل مع النوع الجديد.

C#

```
// مثال على مخالفة OCP (افتراضي)
public class LibraryCatalog
{
    private List<LibraryItem> _items;

    public void AddItem(LibraryItem item)
    {
        _items.Add(item);
        // إذا أردنا إضافة منطق خاص لكل نوع هنا، سنخالف OCP
        // if (item is Book) { /* ... */ }
        // else if (item is EBook) { /* ... */ }
    }

    public void DisplayAllItems()
    {
        foreach (var item in _items)
        {
            // هنا قد نضطر لتعديل الكود عند إضافة نوع جديد
            // if (item is Book) { ((Book)item).DisplayInfo(); }
            // else if (item is EBook) { ((EBook)item).DisplayInfo(); }
            item.DisplayInfo(); // جزئيا OCP هذه الدالة كانت جيدة وتطبق
        }
    }
}
```

المشكلة/المخالفة

إضافة أنواع جديدة من عناصر المكتبة كانت ستتطلب تعديل الكلاسات التي تتعامل مع هذه العناصر، مما يزيد من خطر إدخال أخطاء في الكود الموجود.

الكود بعد التعديل (تطبيق OCP)

تم تصميم الكلاسات الأساسية (`LibraryItem`) والواجهات (`ISearchable` , `IBorrowable` , `IDisplayable`) بحيث يمكن إضافة أنواع جديدة من عناصر المكتبة (مثل `Magazine`) دون تعديل أي كود موجود.

تم إضافة كلاس `Magazine` الذي يرث من `LibraryItem` وينفذ الواجهات المطلوبة، دون الحاجة لتعديل `LibraryCatalog` أو `BorrowingService` .

C#

```
// Magazine.cs (كلاس جديد بدون تعديل الكلاسات القديمة)
namespace LibraryManagementApp.Core
{
    public class Magazine : LibraryItem, IBorrowable, ISearchable
    {
        public int IssueNumber { get; set; }
        public DateTime PublicationDate { get; set; }
        public bool IsBorrowed { get; private set; }

        public Magazine(string title, string author, string isbn, int issueNumb
            : base(title, author, isbn)
        {
            IssueNumber = issueNumber;
            PublicationDate = publicationDate;
            IsBorrowed = false;
        }

        public override void DisplayInfo() { /* ... */ }
        public void BorrowItem() { /* ... */ }
        public void ReturnItem() { /* ... */ }
        public bool Search(string query) { /* ... */ }
    }
}

// LibraryCatalog.cs (لم يتم تعديله لإضافة Magazine)
namespace LibraryManagementApp.Services
{
    public class LibraryCatalog
    {
        private List<LibraryItem> _items;
        public void AddItem(LibraryItem item) { _items.Add(item); }
        // ... باقي الدوال تعمل مع أي LibraryItem جديد
```

```
    }
}
```

التحسينات

- **قابلية التوسع:** يمكن إضافة أنواع جديدة من عناصر المكتبة بسهولة دون الحاجة لتعديل الكود الموجود.
- **تقليل الأخطاء:** يقلل من خطر إدخال أخطاء في الكود الذي يعمل بالفعل.
- **مرونة التصميم:** يجعل النظام أكثر مرونة وقابلية للتكيف مع المتطلبات المستقبلية.

3. مبدأ استبدال ليسكوف (Liskov Substitution Principle - LSP)

الشرح

ينص مبدأ استبدال ليسكوف على أن **"يجب أن تكون الكائنات من الكلاسات المشتقة قابلة للاستبدال بكائنات من الكلاس الأساسي دون تغيير صحة البرنامج"**. بمعنى آخر، إذا كان `S` نوعاً فرعياً من `T` ، فيجب أن تكون الكائنات من النوع `S` قابلة للاستبدال بكائنات من النوع `T` دون تغيير سلوك البرنامج.

الكود الأصلي (قبل التعديل)

في نظام الأسبوع الثالث، كانت جميع الكلاسات المشتقة من `LibraryItem` (مثل `Book` , `EBook` , `AudioBook`) تنفذ `IBorrowable` و `ISearchable` . هذا كان قد يؤدي إلى مشكلة إذا كان هناك نوع من العناصر (مثل `EBook`) لا يمكن استعارته فعلياً، ولكنه كان مجبراً على تنفيذ دوال `BorrowItem()` و `ReturnItem()` .

```C#
// مثال على مخالفة LSP (افتراضي)
// لو كان EBook ينفذ IBorrowable ويقوم بدوال فارغة أو يرمي استثناء
public class EBook : LibraryItem, IBorrowable, ISearchable
{
    // ... خصائص ومشيد
    public void BorrowItem() { /* لا يمكن استعارة كتاب إلكتروني، ماذا نفعل هنا؟ */ }
    public void ReturnItem() { /* ... */ }
    // ...
}
```

المشكلة/المخالفة

إذا كان `EBook` ينفذ `IBorrowable` ولكنه لا يستطيع تقديم سلوك منطقي لدوال `BorrowItem()` و `ReturnItem()` (مثلاً، يرمي استثناء أو لا يفعل شيئاً)، فإن استبدال `LibraryItem` بـ `EBook` في سياق يتوقع `IBorrowable` قد يكسر البرنامج أو يؤدي إلى سلوك غير متوقع.

الكود بعد التعديل (تطبيق LSP)

تم تطبيق LSP من خلال التأكد من أن جميع الكلاسات المشتقة تحافظ على العقد السلوكي للكلاس الأساسي والواجهات التي تنفذها. النقطة الأبرز هنا هي أن `EBook` لم يعد ينفذ `IBorrowable` ، لأنه لا يمكن استعارته.

- `Book` , `AudioBook` , `Magazine` تنفذ `IBorrowable` لأنها قابلة للاستعارة.
- `EBook` لا ينفذ `IBorrowable` لأنه رقمي ومتاح دائماً، وبالتالي لا معنى لدوال الاستعارة والإرجاع له. هذا يضمن أن أي كود يتوقع `IBorrowable` لن يتم تمرير `EBook` إليه بشكل خاطئ.

C#

```
// EBook.cs (بعد تعديل ISP، يلتزم بـ LSP أصبح)
namespace LibraryManagementApp.Core
{
    public class EBook : LibraryItem, ISearchable // لا ينفذ IBorrowable
    {
        // ... خصائص ومشيد
        public override void DisplayInfo() { /* ... */ }
        public bool Search(string query) { /* ... */ }
        // هنا ReturnItem أو BorrowItem لا توجد دوال
    }
}

// LibraryCatalog.cs (يستخدم تعدد الأشكال بشكل صحيح)
namespace LibraryManagementApp.Services
{
    public class LibraryCatalog
    {
        private List<LibraryItem> _items;
        public List<LibraryItem> SearchItems(string query)
        {
            List<LibraryItem> results = new List<LibraryItem>();
            foreach (var item in _items)
            {
                if (item is ISearchable searchableItem) // التحقق من الواجهة
                {
                    if (searchableItem.Search(query)) { results.Add(item); }
                }
            }
            return results;
        }
    }
}
```

التحسينات

- **سلامة البرنامج:** يضمن أن استبدال الكائنات المشتقة بالكائنات الأساسية لن يؤدي إلى سلوك غير متوقع أو أخطاء.
- **تصميم منطقي:** يعكس الواقع بشكل أفضل (الكتاب الرقمي لا يُستعار).
- **تقليل التعقيد:** يزيل الحاجة إلى منطق معقد للتعامل مع حالات خاصة في الكلاسات المشتقة.

4. مبدأ فصل الواجهات (Interface Segregation Principle - ISP)

الشرح

ينص مبدأ فصل الواجهات على أن **"يجب ألا تُجبر الكائنات على تنفيذ واجهات لا تستخدمها"**. بمعنى آخر، من الأفضل أن يكون لدينا العديد من الواجهات الصغيرة والمتخصصة بدلاً من واجهة واحدة كبيرة وعامة.

الكود الأصلي (قبل التعديل)

في نظام الأسبوع الثالث، كانت الواجهات `IBorrowable` و `ISearchable` موجودة، ولكن كان من الممكن أن تكون هناك واجهة واحدة ضخمة `ILibraryItem` تحتوي على جميع الدوال (عرض، استعارة، إرجاع، بحث). هذا كان سيجبر جميع أنواع عناصر المكتبة على تنفيذ جميع هذه الدوال، حتى لو لم تكن منطقية لها (مثل `EBook` الذي لا يُستعار).

C#

```
// مثال على مخالفة ISP (افتراضي)
public interface ILibraryItem // واجهة ضخمة
{
    void DisplayInfo();
    void BorrowItem();
    void ReturnItem();
    bool IsBorrowed { get; }
    bool Search(string query);
}

public class EBook : ILibraryItem // مجبر على تنفيذ BorrowItem و ReturnItem
{
    // ...
    public void BorrowItem() { /* لا معنى لها هنا */ }
    public void ReturnItem() { /* لا معنى لها هنا */ }
    // ...
}
```

المشكلة/المخالفة

إجبار `EBook` على تنفيذ دوال `BorrowItem()` و `ReturnItem()` كان سيؤدي إلى كود غير منطقي أو غير مستخدم، مما يزيد من تعقيد الكلاس دون فائدة.

الكود بعد التعديل (تطبيق ISP)

تم تقسيم الواجهات إلى واجهات صغيرة ومتخصصة، كل منها يمثل سلوكاً محدداً:

- **IDisplayable** : تحتوي على دالة DisplayInfo() .
- **IBorrowable** : تحتوي على دوال BorrowItem() , ReturnItem() , و خاصية IsBorrowed .
- **ISearchable** : تحتوي على دالة Search(string query) .

الآن، كل كلاس ينفذ فقط الواجهات التي يحتاجها:

- LibraryItem الأساسي ينفذ IDisplayable .
- Book , AudioBook , Magazine تنفذ IBorrowable و ISearchable (بالإضافة إلى IDisplayable من LibraryItem).
- EBook ينفذ ISearchable فقط (بالإضافة إلى IDisplayable من LibraryItem)، ولا ينفذ IBorrowable .

C#

```
// LibraryInterfaces.cs
namespace LibraryManagementApp.Core
{
    public interface IDisplayable { void DisplayInfo(); }
    public interface IBorrowable { bool IsBorrowed { get; } void BorrowItem(); 
    public interface ISearchable { bool Search(string query); }
}

// EBook.cs (ينفذ فقط ما يحتاجه)
namespace LibraryManagementApp.Core
{
    public class EBook : LibraryItem, ISearchable // يرث IDisplayable وينفذ ISear
    {
        // ...
        // هنا ReturnItem أو BorrowItem لا توجد دوال
    }
}
```

التحسينات

- **تقليل التعقيد:** الكلاسات أصبحت أبسط لأنها تنفذ فقط الدوال التي تحتاجها.
- **مرونة التصميم:** يسهل إضافة سلوكيات جديدة للنظام دون التأثير على الكلاسات التي لا تحتاجها.
- **تحسين قابلية القراءة:** الكود أصبح أكثر وضوحاً ومنطقية.

5. مبدأ عكس التبعية (Dependency Inversion Principle - DIP)

الشرح

ينص مبدأ عكس التبعية على أن **"يجب أن تعتمد الوحدات عالية المستوى على التجريدات (Abstractions)، وليس على التفاصيل (Concretions). ويجب أن تعتمد التفاصيل أيضاً على التجريدات"**. بمعنى آخر، يجب أن تعتمد الكلاسات على الواجهات أو الكلاسات التجريدية بدلاً من الاعتماد المباشر على الكلاسات الملموسة (Concrete Classes).

الكود الأصلي (قبل التعديل)

في نظام الأسبوع الثالث، كان من الممكن أن يعتمد `BorrowingService` بشكل مباشر على كلاس `EmailNotifier` لإرسال الإشعارات، مما يجعله مرتبطاً بشكل وثيق بتنفيذ معين للإشعارات.

```
C#

// مثال على مخالفة DIP (افتراضي)
public class BorrowingService
{
    // اعتماد مباشر على كلاس ملموس
    private EmailNotifier _emailSender = new EmailNotifier();

    public void BorrowItem(string isbn, int memberId)
    {
        // ... منطق الاستعارة
        _emailSender.SendEmail(member.Email, "You borrowed a book."); // م مباشر
    }
}
```

المشكلة/المخالفة

الاعتماد المباشر على `EmailNotifier` يجعل `BorrowingService` مرتبطاً بشكل وثيق بهذا النوع المحدد من الإشعارات. إذا أردنا تغيير طريقة الإشعار (مثلاً، استخدام SMS أو Push Notification)، سنضطر لتعديل `BorrowingService` ، مما يخالف مبدأ OCP أيضاً.

الكود بعد التعديل (تطبيق DIP)

تم تطبيق DIP من خلال جعل `BorrowingService` يعتمد على واجهة `INotificationService` بدلاً من كلاس `EmailNotifier` الملموس.

- تم تعريف واجهة `INotificationService` .
- تم إنشاء تنفيذات ملموسة لهذه الواجهة مثل `PushNotifier` , `SmsNotifier` , `EmailNotifier` .
- عبر المشيد (Constructor Injection)، `INotificationService` يستقبل كائناً من نوع `BorrowingService` مما يعني أنه لا يعرف أي شيء عن التنفيذ الفعلي للإشعار.

```
C#
```

```
// NotificationService.cs (الواجهة والتنفيذات)
namespace LibraryManagementApp.Services
{
    public interface INotificationService
    {
        void SendNotification(string recipient, string message);
    }

    public class EmailNotifier : INotificationService { /* ... */ }
    public class SmsNotifier : INotificationService { /* ... */ }
    public class PushNotifier : INotificationService { /* ... */ }
}

// BorrowingService.cs (يعتمد على التجريد)
namespace LibraryManagementApp.Services
{
    public class BorrowingService
    {
        private INotificationService _notificationService; // الاعتماد على الواجهة

        public BorrowingService(LibraryCatalog catalog, MemberService memberSer
        {
            // ...
            _notificationService = notificationService;
        }

        public void BorrowItem(string isbn, int memberId)
        {
            // ... منطق الاستعارة
            _notificationService.SendNotification(member.Email, "You borrowed a
        }
    }
}

// Program.cs (حقن التبعية)
namespace LibraryManagementApp
{
    class Program
    {
        static void Main(string[] args)
        {
            INotificationService emailNotifier = new EmailNotifier();
            BorrowingService borrowingService = new BorrowingService(catalog, m
            // ...
        }
    }
}
```

التحسينات

- **قابلية التوسع:** يمكن تغيير خدمة الإشعارات المستخدمة بسهولة (من Email إلى SMS مثلاً) دون تعديل `BorrowingService` .
- **قابلية الاختبار:** يسهل اختبار `BorrowingService` عن طريق تمرير كائنات وهمية (Mock Objects) لـ `INotificationService` .
- **تقليل الارتباط:** يقلل من الارتباط بين الكلاسات، مما يجعل النظام أكثر مرونة.

الخلاصة

من خلال تطبيق مبادئ SOLID الخمسة، تم تحويل نظام إدارة المكتبة من مجرد مجموعة من الكلاسات الوظيفية إلى نظام مصمم بشكل جيد، مرن، قابل للصيانة، وقابل للتوسع. كل مبدأ ساهم في تحسين جانب معين من جودة الكود، مما أدى إلى نظام برمجي أكثر قوة واستدامة.